Governed Automation Starter: Build Blueprint

Version 1.1, October 2026 (adds section 18, implementation guide for coding agents). A gateway that puts spend limits, human approval, an audit trail and a kill switch around any AI automation.

0. Read this first: what "works 100%" can honestly mean

No governance layer works in 100% of setups. Research shows why: a gateway only governs traffic that passes through it, and direct API calls, browser-driven agents and embedded SaaS connectors can bypass it. So this blueprint targets something verifiable instead: **every supported path is enforced, every unsupported path is detected.**

Three coverage tiers define the promise:

Tier	How it works	Guarantee
1. Enforced	Traffic goes through the gateway (base URL swap, virtual key)	Budgets, approvals, kill switch, audit are enforced before the call
2. Gated actions	Risky tools (email, payments, DB writes) call a check endpoint and verify a signed decision	The action only runs with a valid approval token
3. Observed	Provider usage APIs are polled and compared with gateway records	Unrouted spend is detected within a day and flagged, not blocked

Anything outside these tiers (local models with no network hop, vendor-hosted agents you cannot reroute) is documented as unsupported, not hidden.

1. Goals and non-goals

Goals: per-client and per-workflow budgets; approval for risky actions; tamper-evident audit log; kill switch and loop breakers; client-facing spend and ROI reports; works with n8n, Python and JS SDKs, LangChain-style frameworks and Claude, OpenAI and Gemini APIs.

Non-goals (v1): prompt-injection defense, content moderation, model routing and fallback, SSO, billing, a custom policy language. Use existing tools where needed.

2. Requirements

Functional

- Issue virtual keys per client and workflow; real provider keys never leave the gateway.
- Enforce monthly and per-run budgets, rate limits, allowed models and risk class.
- Hold risky actions until a human approves or denies; record who and when.
- Write an append-only, hash-chained event for every call and decision.
- Show spend, pending approvals, alerts and a monthly report per client.

Non-functional

- Added latency per call: target under 50 ms excluding provider time (a goal to measure, not a measured result).

- Stateless gateway replicas; Postgres as the single source of truth.
- Configurable fail mode per workflow: fail-closed for high risk, fail-open with alert for low risk.
- Retention configurable per client (see section 9).

Constraints: solo developer, part-time; reuse open-source components; no custody of client money.

3. Compatibility strategy (the "universal" part)

3.1 One endpoint per native dialect, no cross-translation in v1

Open-source gateways commonly translate between OpenAI, Anthropic and Gemini formats, but translation is where bugs live. One public issue on a translating gateway shows streaming tool calls being converted into broken events. So v1 exposes each provider's own dialect and passes it through, with metering added:

Dialect	Endpoint	Auth header the gateway must accept
OpenAI Chat Completions	`/v1/chat/completions`	`Authorization: Bearer`
OpenAI Responses	`/v1/responses`	`Authorization: Bearer`
OpenAI embeddings	`/v1/embeddings`	`Authorization: Bearer`
Anthropic Messages	`/v1/messages`	`x-api-key`
Gemini	`/v1beta/models/{model}:generateContent` and `:streamGenerateContent`	`x-goog-api-key`
Model list	`GET /v1/models`	all

`GET /v1/models` matters: n8n validates its credential by calling it. Cross-provider translation is a later optional module, behind a flag, with its own test suite.

3.2 Client compatibility matrix

Client	How to connect	Notes
n8n	OpenAI credential with Base URL set to the gateway, including `/v1`	Turn off Responses API mode on the chat model node if your build does not implement it. n8n also has built-in human-in-the-loop for agent tool calls
OpenAI SDKs (Python, JS)	`base_url`	Works as is
Anthropic SDKs, Claude Code	`base_url` / `ANTHROPIC_BASE_URL`	Gateway must accept `x-api-key`

Gemini SDK	custom endpoint	Weakest standard for base URL overrides; test per SDK version
LangChain, LlamaIndex, CrewAI	provider `base_url` argument	Same as the underlying SDK
Raw HTTP	any language	Fully supported
Realtime, voice, image, video APIs	not in v1	Meter later or document as unsupported

Attribution rule that works everywhere: **one virtual key per (client, workflow).** Optional headers (`X-Run-Id`, `X-Workflow`) add detail where the client can set them.

3.3 Streaming and usage

- Streamed responses are forwarded as they arrive; usage is read from the final event. For OpenAI-style streams, the gateway injects the option that includes usage in the stream.
- If a client disconnects mid-stream and usage never arrives, record an estimate flagged `estimated=true` and let reconciliation correct it.
- Token fields differ by provider. Cached tokens are the classic trap: OpenTelemetry's convention says input tokens should include cached ones, so naive totals overlap. Store the raw provider usage JSON plus normalized fields: `input`, `output`, `cache_read`, `cache_write`, `reasoning`.

3.4 Telemetry standard

OpenTelemetry's GenAI conventions are still at "Development" status as of late 2026 (all attributes), and they moved to a separate repository in June 2026. So: use your own stable internal schema and add an OpenTelemetry exporter that maps to `gen_ai.*` names. Do not make the internal schema depend on them.

3.5 Build on LiteLLM or write your own?

LiteLLM is MIT-licensed, supports 100+ providers, and has virtual keys, budgets, rate limits and Postgres-backed spend tracking. Its audit logs and SSO are in the paid tier. A reported bug showed budgets being bypassed for certain model-naming setups, and some job types cannot be priced and are rejected. Lessons:

1. Use it as the **routing layer** if you want speed, and put your own governance service in front or beside it (approvals, audit chain, reports, kill switch).
2. Never trust one counter. Run independent reconciliation (section 7).
3. Pin versions and review dependencies; gateway packages are a supply-chain target.

Decision for v1: **thin own gateway (FastAPI) for the three native dialects, with LiteLLM optional later for exotic providers.** This keeps control of the parts that are your product and avoids inheriting a large codebase you cannot fully verify.

4. Architecture

```
Client automation ──> Gateway (FastAPI, stateless, N replicas)
                        |  Auth: virtual key -> client, workflow, policy
```

```
                        ├─ Auth: virtual key -> client, workflow, policy
                        ├─ Budget: atomic reserve -> settle
                        ├─ Policy: model allowlist, rate limit, risk class
                        ├─ Kill switch + loop breaker
                        └─ Forward -> Provider (Claude / OpenAI / Gemini)
Gateway ──> Postgres (keys, policy, spend, runs, approvals, audit chain)
Gateway ──> Approval service ──> Telegram / email / webhook / n8n
Tool wrappers ──> Check endpoint ──> signed decision token
Reconciler (cron) ──> Provider usage APIs ──> drift alerts
Dashboard (React/Next.js) ──> Admin API
```

Reference stack: Python 3.12, FastAPI, httpx (streaming), Postgres (Supabase is fine), Redis (optional, for rate limits), React or Next.js with Tailwind, Docker Compose, Alembic migrations, pytest, Playwright for dashboard tests.

5. Request lifecycle

1. **Authenticate:** hash the presented key, look up client, workflow, policy. Unknown key: 401.
2. **Kill check:** workflow, client or global flag set: 503 with a clear message.
3. **Policy check:** model allowed? rate limit ok? run step and cost ceilings ok?
4. **Reserve budget atomically:** estimate worst-case cost (input tokens counted, output capped by `max_tokens`) and run `UPDATE budgets SET reserved = reserved + :est WHERE spent + reserved + :est <= cap RETURNING ...`. No row returned means over budget: 429. Atomic SQL prevents two parallel calls from both slipping under the cap.
5. **Forward** to the provider using the stored provider key.
6. **Settle:** replace the reservation with actual cost from usage; write the audit event.
7. **Loop breaker:** the same run exceeding step, call-rate or cost-spike limits trips the kill flag for that run and alerts.

Approvals are **not** inserted into every LLM call (that would paralyze workflows). They gate **actions**, section 6.

6. Policy and approvals

Risk classes (per workflow):

Class	Example	Default control
R0 read-only	summarize, classify	Budget and log only
R1 internal write	update CRM field	Log; sample review
R2 external-facing	email to a customer	Approval required
R3 money or irreversible	payment, deletion	Approval plus two-person rule (optional)

Controls are matched to the class, not applied uniformly.

Approval flow for actions:

1. The tool wrapper (or an n8n node) calls `POST /v1/actions/check` with the proposed action.
2. The gateway returns `allow` (R0/R1), `deny`, or `pending` with an `approval_id`.
3. The [illegible] with the action context and Approve/Deny buttons (Telegram bot

3. The approver gets a message with the action, context and Approve/Deny buttons (Telegram bot, email link, or n8n's own approval step).
4. On approval the gateway issues a **signed, short-lived decision token** bound to the action hash and requester. The executor verifies it before acting.
5. Everything is logged: proposer, approver, timestamps, action hash.

Design notes from current practice: MCP itself does not standardize approval, so approval state is a gateway convention. Approval is not authorization; permanent prohibitions belong in access policy, not repeated human clicks. Human approval only adds safety when the human sees information the static policy does not, so show the approver the actual content, not just a button.

Policy example (per workflow):

```
{
  "workflow": "invoice-followup",
  "risk_class": "R2",
  "models": ["claude-*", "gpt-*"],
  "budget": { "monthly_usd": 40, "per_run_usd": 0.50 },
  "limits": { "max_steps": 20, "rpm": 30 },
  "approval": { "required_for": ["send_email"], "timeout_min": 120, "on_timeout": "deny"
  "fail_mode": "closed"
}
```

7. Cost accounting and reconciliation

- **Price table:** versioned rows per model (`input`, `output`, `cache_read`, `cache_write`, `reasoning` per million tokens, `effective_from`). A model with no price: reject in strict mode, or charge a configurable fallback and flag it.
- **Daily reconciliation:** poll provider organization usage and cost APIs (Anthropic and OpenAI use separate admin-tier keys; cost data is daily granularity) and compare with gateway totals per day and model. Alert on drift above a threshold (for example 3%).
- **Attribution caveat:** one source says Anthropic's usage report has no per-API-key filter, so use one provider workspace per client where you need exact attribution. Another says Gemini has no equivalent usage API, so Gemini stays gateway-only. Verify both against current provider docs before relying on them.
- **Unrouted spend:** drift that gateway records cannot explain means traffic is bypassing the gateway. Surface it as "possible bypass" in the dashboard. This is Tier 3.

8. Data model

Table	Key fields
`clients`	id, name, status, retention_days, timezone
`workflows`	id, client_id, name, risk_class, policy_json, status
`virtual_keys`	id, workflow_id, key_hash, prefix, created_at, revoked_at
`provider_credentials`	id, client_id, provider, encrypted_key, workspace_ref
`budgets`	workflow_id, period, cap_usd, spent_usd, reserved_usd

`runs`	id, workflow_id, started_at, ended_at, status, step_count, cost_usd
`events`	id, run_id, ts, type, provider, model, usage_json, cost_usd, estimated, content_ref, prev_hash, hash
`approvals`	id, run_id, action_hash, proposed_by, status, approver, decided_at, expires_at
`prices`	model, input, output, cache_read, cache_write, reasoning, effective_from
`reconciliations`	day, provider, model, gateway_usd, provider_usd, drift_pct
`users`	id, email, role, client_scope

9. Audit log, privacy and retention

- **Tamper evidence:** each `events` row stores `hash = H(prev_hash || canonical_json(row))`. A nightly job verifies the chain and optionally publishes the latest hash to an external store, so silent edits become detectable.
- **Content handling:** by default store hashes plus redacted excerpts; full prompts and responses are opt-in per client, encrypted, with a retention period.
- **Retention:** EU AI Act rules for high-risk systems set a six-month minimum for logs, and logs that can be altered lose evidential value. Not every client workflow is high-risk and application dates have shifted, so make retention configurable and check with a lawyer per client. GDPR data minimization pulls the other way, so keep personal data out of logs where you can.
- **Human-oversight records:** log who approved and when. This supports "real oversight, not rubber-stamping" if a client is ever audited.

10. Security essentials

- Provider keys encrypted at rest (envelope encryption; keys from environment or a secrets manager, never in the database in plaintext).
- Virtual keys shown once, stored hashed, revocable instantly.
- Strict tenant isolation: every query scoped by `client_id`; test it explicitly.
- Admin API behind login with roles; no admin routes on the proxy port.
- Rate limits on the proxy and on approval endpoints; approval links single-use and expiring.
- Treat all model output and prompt content as untrusted when rendering in the dashboard.
- Dependency pinning, lockfiles, automated vulnerability scanning in CI.

11. API surface

Endpoint	Purpose
Proxy dialect endpoints (section 3.1)	Metered pass-through
`POST /v1/actions/check`	Ask for allow, deny or pending on an action
`GET /v1/approvals/{id}`	Poll approval state

`GET /v1/approvals/{id}`	Poll approval state
`POST /v1/approvals/{id}/decision`	Approve or deny (approver auth)
`POST /admin/keys`, `DELETE /admin/keys/{id}`	Manage virtual keys
`POST /admin/kill`	Kill workflow, client or global
`GET /admin/spend`, `GET /admin/reports/{client}/{month}`	Spend and reports

12. Repository layout

```
gov-gateway/
  gateway/        # FastAPI proxy, dialect adapters, streaming
  policy/         # risk classes, budget reserve/settle, loop breaker
  approvals/      # approval service, Telegram/email/webhook channels
  audit/          # hash chain, verifier, export
  reconcile/      # provider usage pollers, drift detection
  admin_api/      # keys, kill switch, reports
  dashboard/      # React or Next.js app
  sdk/            # tiny wrappers: check_action() for Python and JS
  tests/          # contract, race-condition, tenancy tests
  docker-compose.yml
```

13. Build plan (rough estimates for part-time solo work)

Phase	Scope	Exit test
1. Skeleton (about 1 week)	Auth, OpenAI Chat endpoint, budget reserve/settle, audit chain, one kill flag	n8n workflow runs through the gateway and stops at its cap
2. Dialects (about 1 week)	Anthropic Messages, Responses, embeddings, Gemini, streaming usage	Contract tests pass for each SDK
3. Approvals (about 1 week)	Action check, Telegram approval, decision tokens, SDK wrappers	Risky action blocked until approved, token verified
4. Reconcile and dashboard (about 1 to 2 weeks)	Price table, daily reconciliation, spend and approvals UI, monthly report	Drift alert fires on a deliberately unrouted call
5. Pilot (ongoing)	1 to 2 real clients, fix what breaks	Written client feedback and a measured savings number

14. Testing: how you earn "compatible"

- **Contract tests:** for each client in the matrix (n8n, OpenAI SDK, Anthropic SDK, Gemini SDK, LangChain), run a script covering plain call, streaming, tool call, error, and long output. Pin SDK versions and rerun on upgrades.
- **Race tests:** hundreds of parallel calls against a tiny budget; total spend must never exceed the

cap.

- **Failure tests:** provider timeout, malformed stream, client disconnect mid-stream, database outage in both fail modes.
- **Tenancy tests:** a key from client A must never read or spend client B's data.
- **Bypass test:** make a call outside the gateway; reconciliation must flag it.
- **Soak test:** a loop that calls endlessly must be stopped by the loop breaker.

15. Trade-offs and what to revisit

Decision	Benefit	Cost	Revisit when
Native dialects, no translation	Fewer streaming and tool-call bugs	Clients cannot swap providers by changing one string	Clients ask for provider fallback
Thin own gateway vs LiteLLM	Control and small surface	You maintain adapters	Provider count grows past three
Reserve then settle	Cap cannot be overshot by races	Brief over-reservation slows near-cap calls	Calls get long or expensive
Approvals on actions only	Workflows stay fast	Needs tool wrappers	Clients want call-level review
Own schema, OTel export	Stable under shifting standards	Mapping code	OTel GenAI reaches stable
Hash chain in Postgres	Simple, cheap	Weaker than external anchoring	A client needs regulator-grade proof

16. Open questions to verify before building

1. Current provider terms: do your clients' provider agreements allow routing through your gateway with your key-handling model?
2. Gemini SDK base-URL override behavior in the versions your clients use.
3. Exact Anthropic and OpenAI usage and cost API fields and attribution options today.
4. Local legal requirements for logs and data location for your first clients.
5. Whether n8n's current approval node and your approval API can be unified or should stay separate.
6. Real demand: the 10-conversation test from earlier. This blueprint does not replace it.

17. Definition of done for v1

A client's n8n or Python automation runs through the gateway; it stops at its budget; a risky action waits for approval and carries a verified token; the audit chain verifies; a deliberately unrouted call is flagged within a day; and the client receives a monthly report showing spend and results.

---

18. Implementation guide for a coding agent

Sections 0 to 17 explain the design. This section tells an AI coding agent (or a human developer) exactly what to build, in what order, and how to prove each step works. Where this section conflicts with sections 0 to 17, this section wins; record the conflict in `docs/decisions.md`.

18.1 Operating rules

1. Build phases in order. Do not start a phase until the previous phase's acceptance tests pass.
2. Build only what is listed. The non-goals in section 1 stay out.
3. Every task ends with tests green and one commit with a message of the form `T1.4: virtual key auth`.
4. Never hardcode secrets. Never log provider keys or full virtual keys (log the 8-character prefix only).
5. **VERIFY** means: read the provider's current official documentation, save a real recorded response as a test fixture in `tests/fixtures/`, and write the code against the fixture, not against memory.
6. **DECISION** means: stop and ask the human. Do not guess.
7. Money is stored as integer micro-USD (1 USD = 1,000,000) everywhere. No floats for money.
8. All timestamps are UTC (`timestamptz`).
9. Add no dependency without a one-line reason in `docs/decisions.md`. Pin all versions in the lockfile.
10. Stop and report if a test cannot be made to pass after three honest attempts; do not weaken the test.

18.2 Fixed technical choices

Area	Choice
Language	Python 3.12
Web	FastAPI, uvicorn
HTTP client	httpx (async, streaming)
Database	Postgres 16, SQLAlchemy 2.x async with asyncpg, Alembic migrations
Validation	pydantic v2
Crypto	`cryptography` (AES-GCM) for provider keys; `hmac` and `hashlib` for the rest
Tests	pytest, pytest-asyncio, respx; real OpenAI, Anthropic and Gemini SDKs for contract tests only
Packaging	`uv` with a lockfile
Local run	Docker Compose: `postgres`, `gateway`, `mock-provider` (dashboard added in phase 4)
Lint	ruff and mypy in CI

18.3 Bootstrap

Create the layout from section 12. Environment variables (document in `.env.example`, never commit `.env`):

Variable	Meaning
`DATABASE_URL`	Postgres connection string
`MASTER_ENCRYPTION_KEY`	32 random bytes, base64; encrypts provider keys
`DECISION_SIGNING_KEY`	32 random bytes, base64; signs approval tokens
`ADMIN_TOKEN`	Bearer token for the admin API until phase 4 adds login
`UPSTREAM_TIMEOUT_S`	Provider timeout, default 120
`DEFAULT_FAIL_MODE`	`closed` or `open`, default `closed`
`DEFAULT_MAX_OUTPUT_TOKENS`	Assumed output size when the request sets none, default 4096

Virtual key format: `gk_` + 32 URL-safe random characters. Store only `sha256(key)` and the first 8 characters. Show the full key once at creation.

18.4 Database schema (migration 0001)

```
CREATE TABLE clients (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL, status text NOT NULL DEFAULT 'active',
  retention_days int NOT NULL DEFAULT 180, created_at timestamptz NOT NULL DEFAULT now())

CREATE TABLE workflows (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  client_id uuid NOT NULL REFERENCES clients(id), name text NOT NULL,
  risk_class text NOT NULL CHECK (risk_class IN ('R0','R1','R2','R3')),
  policy jsonb NOT NULL DEFAULT '{}', status text NOT NULL DEFAULT 'active',
  UNIQUE (client_id, name));

CREATE TABLE virtual_keys (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  workflow_id uuid NOT NULL REFERENCES workflows(id),
  key_hash bytea NOT NULL UNIQUE, prefix text NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now(), revoked_at timestamptz);

CREATE TABLE provider_credentials (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  client_id uuid NOT NULL REFERENCES clients(id),
  provider text NOT NULL CHECK (provider IN ('openai','anthropic','gemini')),
  label text NOT NULL DEFAULT 'default', ciphertext bytea NOT NULL, nonce bytea NOT NULL,
  UNIQUE (client_id, provider, label));

CREATE TABLE budgets (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  workflow_id uuid NOT NULL REFERENCES workflows(id),
  period text NOT NULL,                       -- 'YYYY-MM'
  cap_micro bigint NOT NULL,
  spent_micro bigint NOT NULL DEFAULT 0
```

```
  spent_micro bigint NOT NULL DEFAULT 0,
  reserved_micro bigint NOT NULL DEFAULT 0,
  UNIQUE (workflow_id, period),
  CHECK (spent_micro >= 0 AND reserved_micro >= 0));

CREATE TABLE runs (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  workflow_id uuid NOT NULL REFERENCES workflows(id), external_run_id text,
  started_at timestamptz NOT NULL DEFAULT now(), ended_at timestamptz,
  status text NOT NULL DEFAULT 'running', step_count int NOT NULL DEFAULT 0,
  cost_micro bigint NOT NULL DEFAULT 0);

CREATE TABLE events (
  id bigserial PRIMARY KEY, client_id uuid NOT NULL, run_id uuid,
  ts timestamptz NOT NULL DEFAULT now(), type text NOT NULL,
  provider text, model text, usage jsonb, cost_micro bigint,
  estimated boolean NOT NULL DEFAULT false, content_ref text,
  prev_hash bytea, hash bytea NOT NULL);
CREATE INDEX events_client_id_idx ON events (client_id, id);

CREATE FUNCTION events_immutable() RETURNS trigger AS $$
BEGIN RAISE EXCEPTION 'events is append-only'; END; $$ LANGUAGE plpgsql;
CREATE TRIGGER events_no_change BEFORE UPDATE OR DELETE ON events
  FOR EACH ROW EXECUTE FUNCTION events_immutable();

CREATE TABLE kill_flags (
  scope text NOT NULL CHECK (scope IN ('global','client','workflow')),
  scope_id text NOT NULL DEFAULT '', active boolean NOT NULL, reason text,
  set_at timestamptz NOT NULL DEFAULT now(), PRIMARY KEY (scope, scope_id));

CREATE TABLE prices (          -- micro-USD per 1,000,000 tokens
  provider text NOT NULL, model text NOT NULL, effective_from date NOT NULL,
  input bigint NOT NULL, output bigint NOT NULL,
  cache_read bigint NOT NULL DEFAULT 0, cache_write bigint NOT NULL DEFAULT 0,
  reasoning bigint NOT NULL DEFAULT 0, PRIMARY KEY (provider, model, effective_from));

CREATE TABLE approvals (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  workflow_id uuid NOT NULL REFERENCES workflows(id), run_id uuid,
  action_name text NOT NULL, action_hash bytea NOT NULL, action_summary text NOT NULL,
  status text NOT NULL DEFAULT 'pending' CHECK (status IN ('pending','approved','denied',
  approver text, decided_at timestamptz, expires_at timestamptz NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now());
```

Later migrations add `reconciliations` and `users` (phase 4).

18.5 Core algorithms (implement exactly)

Budget reserve, settle, release. Reservation must be one atomic statement so parallel calls cannot overshoot the cap:

```
RESERVE = """UPDATE budgets SET reserved_micro = reserved_micro + :est
 WHERE workflow_id = :wf AND period = :period
   AND spent_micro + reserved_micro + :est <= cap_micro RETURNING id"""
SETTLE  = """UPDATE budgets SET reserved_micro = reserved_micro - :est,
 spent_micro = spent_micro + :actual WHERE id = :id"""
RELEASE = """UPDATE budgets SET reserved_micro = reserved_micro - :est WHERE id = :id"""
```

No row returned from `RESERVE` means over budget: return the dialect-native 429. If `actual > est`, still settle the actual amount and write an event of type `budget_overrun`. Always run `RELEASE` or `SETTLE` in a `finally` block so a crash cannot leave a reservation stuck; a nightly job also clears reservations older than one hour.

Reservation estimate. `est = ceil(est_input_tokens * price.input / 1e6) + ceil(max_output * price.output / 1e6)`, where `est_input_tokens = ceil(len(request_text) / 3)` (deliberately pessimistic) and `max_output` is the request's own limit or `DEFAULT_MAX_OUTPUT_TOKENS`. If no price exists for the model: strict mode returns 400 `model_not_priced`; lenient mode uses a configured fallback price and flags the event.

Cost.

```
def cost_micro(u, p):   # all Usage fields are ints; p fields are micro-USD per 1M tokens
    return ceil((u.input*p.input + u.output*p.output + u.cache_read*p.cache_read
                 + u.cache_write*p.cache_write + u.reasoning*p.reasoning) / 1_000_000)
```

Normalization rule: `Usage.input` must **exclude** cached tokens, so nothing is billed twice. VERIFY per provider how cached and reasoning tokens are reported (for example, whether the provider's reported input count already includes cached tokens) and write one normalization function per adapter with a fixture test.

Audit hash chain (one chain per client).

```
def event_hash(prev: bytes, row: dict) -> bytes:
    payload = json.dumps(row, sort_keys=True, separators=(",", ":"), default=str).encode(
    return hashlib.sha256(prev + payload).digest()
```

Append inside one transaction: `SELECT pg_advisory_xact_lock(hashtext(:client_id))`, read the latest `hash` for the client (use 32 zero bytes if none), compute, insert. The CLI `python -m audit.verify --client <id>` recomputes the whole chain and exits non-zero on the first mismatch.

Adapter interface. One class per dialect, same interface, so the proxy core never contains provider-specific code:

```
class Adapter(Protocol):
    name: str                       # 'openai' | 'anthropic' | 'gemini'
    path_patterns: list[str]
    def extract_key(self, headers: Mapping[str, str]) -> str | None: ...
    def parse(self, path: str, body: dict) -> ParsedRequest: ...      # model, stream, ma
    def build_upstream(self, parsed, body, credential: str) -> UpstreamRequest: ...
    def usage_from_response(self, body: dict) -> Usage: ...
    def usage_from_stream_event(self, event: dict, acc: Usage) -> Usage: ...
    def native_error(self, status: int, kind: str, message: str) -> tuple[int, dict]: ...
```

- Key extraction: OpenAI reads `Authorization: Bearer`; Anthropic reads `x-api-key`; Gemini reads `x-goog-api-key` (or `?key=`). VERIFY each.
- Errors must use the dialect's own error JSON shape so official SDKs raise sensible exceptions. VERIFY the shapes from provider docs and store them as fixtures.

- Streaming: forward upstream bytes to the client unchanged and unbuffered; parse a copy of each SSE event for usage. OpenAI streams do not report usage unless the request asks for it, so the OpenAI adapter adds the usage option to the upstream request when absent and strips the extra usage chunk only if the client did not ask for it. VERIFY exact behavior with a recorded fixture. Anthropic and Gemini report usage inside stream events; VERIFY event names and fields.
- If the stream ends without usage (disconnect, upstream cut): settle with the estimate, set `estimated = true`.

Kill check order (cheapest first): global flag, client flag, workflow flag, loop breaker. Cache flags in process for at most 5 seconds. Kill responses use the dialect-native error with HTTP 503.

Loop breaker. Per run (identified by `X-Run-Id` if present, else by workflow plus a 10-minute window): trip when `step_count > policy.max_steps`, or calls per minute exceed `policy.rpm`, or run cost exceeds `policy.per_run_usd`. Tripping sets the workflow's `kill_flags` row with a reason, writes an event of type `loop_breaker`, and alerts.

Fail mode. If Postgres is unreachable: workflows with `fail_mode = closed` get 503; `open` workflows pass through and write to a local on-disk queue that replays into `events` when the database returns (mark those events `estimated = true`).

Decision token (phase 3). `token = base64url(payload) + "." + base64url(HMAC_SHA256(DECISION_SIGNING_KEY, payload))` with payload `{approval_id, action_hash, workflow_id, exp}`; lifetime at most 15 minutes. The SDK's `verify_token(token, action)` recomputes the action hash, checks the signature, expiry and match, and is single-use via an `approval_used` check in the check endpoint.

18.6 Tasks and acceptance criteria

Phase 1: skeleton

ID	Task	Acceptance
T1.1	Repo, `pyproject`, lockfile, Compose with Postgres, CI running ruff, mypy, pytest	`docker compose up` starts; CI green on an empty test
T1.2	Migration 0001 and SQLAlchemy models	Migration applies and rolls back; `UPDATE events` raises an error
T1.3	Admin API (bearer `ADMIN_TOKEN`): create client, workflow, budget, provider credential (encrypt on write), virtual key	Key shown once; DB holds hash only; credential ciphertext is not plaintext
T1.4	Virtual key auth and policy lookup	Unknown or revoked key: 401 in dialect error shape; valid key resolves client and workflow
T1.5	OpenAI Chat Completions adapter, non-streaming, via `mock-provider`	Response body is byte-identical to the mock's; usage parsed
T1.6	Reserve, settle, release (18.5)	Call over cap returns 429; spent equals sum of actual costs
T1.7	Audit chain and verifier CLI	Verifier passes; editing a row in the

		database (with the trigger disabled) makes it fail
T1.8	Kill flags (global, client, workflow)	Setting a flag makes the next call 503 within 5 seconds; clearing restores service
T1.9	OpenAI streaming passthrough with usage capture	Streamed output identical to the mock's; usage settled; mid-stream disconnect settles an estimate
T1.10	`GET /v1/models`	n8n credential test succeeds (manual check)

Phase 1 exit: an n8n workflow using the gateway as its OpenAI Base URL runs, then is stopped by its budget.

Phase 2: dialects and robustness

ID	Task	Acceptance
T2.1	Anthropic Messages adapter, streaming and non-streaming	Official Anthropic SDK completes calls against the gateway
T2.2	OpenAI Responses and embeddings endpoints	Official OpenAI SDK completes both
T2.3	Gemini adapter (`generateContent` , `streamGenerateContent`)	Official Gemini SDK or plain HTTP completes calls (VERIFY base-URL override)
T2.4	Dialect-native errors for every failure (auth, budget, kill, policy, upstream)	Each SDK surfaces a clear error, not a parse failure
T2.5	Model allowlist and RPM limits from policy	Disallowed model: 403; over-limit: 429 with `Retry-After`
T2.6	Loop breaker	Soak test (endless loop) is stopped within the configured limits
T2.7	Contract test suite (section 14) run in CI against `mock-provider`	All matrix rows pass for plain, streaming, tool call, error, long output

Phase 3: approvals

ID	Task	Acceptance
T3.1	`POST /v1/actions/check` , `GET /v1/approvals/{id}` , `POST /v1/approvals/{id}/decision`	R0/R1: allow; R2/R3: pending; unknown action on a strict workflow: deny
T3.2	Telegram approval channel (DECISION: human supplies the bot token and approver chat IDs)	Approver sees action summary and content; Approve and Deny work; links are single-use

T3.3	Email-link and webhook channels	Webhook payload is signed; replayed decisions are rejected
T3.4	Decision tokens and `check_action()` plus `verify_token()` in Python and JS SDKs	Tampered token or altered action fails verification
T3.5	Expiry job	Pending approvals past `expires_at` become `expired`; `on_timeout` policy applied
T3.6	Audit events for proposed, approved, denied, expired	Chain verifies after a full approval cycle

Phase 4: reconcile and dashboard

ID	Task	Acceptance
T4.1	Price table admin and seed script (VERIFY current prices from official pages)	Every supported model has a price row
T4.2	Reconciler for provider usage and cost APIs (DECISION: human supplies admin-tier keys; VERIFY endpoints and fields)	Daily job stores `gateway` and `provider` totals per model
T4.3	Drift alerts and "possible bypass" status	A call made outside the gateway triggers a flag within one run of the job
T4.4	Dashboard: login with roles, spend by client and workflow, pending approvals, alerts, kill buttons	Playwright test: create key, make call, see spend, kill workflow
T4.5	Monthly report export (CSV and printable HTML) with a field for hours saved	Report totals equal database totals

Phase 5: pilot hardening. Run with one or two real clients; fix defects; write `docs/runbook.md` (backup and restore, key rotation, incident steps, how to read the audit chain); record the measured savings figure.

18.7 Test harness

- `mock-provider`: a small FastAPI app that implements all three dialects and can be scripted per request via a header: `normal`, `slow`, `rate_limit`, `server_error`, `malformed_stream`, `cut_stream`, `tool_call`. It must emit realistic usage fields recorded from real responses.
- **Race test:** set a cap that allows exactly 50 calls; fire 200 concurrent calls; assert `spent_micro + reserved_micro <= cap_micro`, exactly 50 succeed, and the other 150 receive 429.
- **Tenancy test:** a key for client A must never read, spend, kill or approve anything of client B (test every admin and proxy route).
- **Chain test:** after 1,000 mixed events, the verifier passes; flipping one byte anywhere makes it fail.
- **Failure tests:** database down in both fail modes; provider timeout; client disconnect mid-stream.

18.8 Master prompt to hand to the agent

18.8 Master prompt to hand to the agent

Paste this at the start of an agent session together with the whole blueprint:

> You are implementing the Governed Automation Starter described in this document. Read sections 0 to 18 fully before writing code. Follow section 18.1 as binding rules. Work phase by phase and task by task in the order of section 18.6. Before each task, state which task ID you are doing; after each task, run the tests and show the result before committing. Use only the technical choices in 18.2 and the schema and algorithms in 18.4 and 18.5. When a task says VERIFY, read the official provider documentation and record real response fixtures first. When a task says DECISION, stop and ask me. Do not add features outside the task list. If you cannot make a test pass after three honest attempts, stop and report what you tried; never weaken or delete a test to get green.

18.9 Completion checklist for v1

- ⬛ All phase 1 to 4 acceptance tests pass in CI.
- ⬛ An n8n workflow and a Python script both run through the gateway end to end.
- ⬛ Official OpenAI, Anthropic and Gemini SDK contract tests pass.
- ⬛ Race, tenancy, chain and failure tests pass.
- ⬛ A risky action is blocked until approved and verified with a decision token.
- ⬛ A deliberately unrouted provider call appears as "possible bypass" within a day.
- ⬛ `.env.example`, `docs/decisions.md` and `docs/runbook.md` exist; no secrets in the repository.
- ⬛ A client receives a monthly report whose totals match the database.

Honest caveat: this guide is a design, not tested code. Provider formats, prices and admin APIs change; every VERIFY item exists because the correct answer must come from current official documentation, not from this document.